

BLOC GÉNIE LOGICIEL

PROJET EASYSAVE

CAHIER DE CHARGE

VERSION 1.0

ÉCOLE D'INGÉNIEURS – CESI

Cycle Ingénieur A3 – Génie Logiciel

Contents

1. Présentation du projet	3
1.1 Contexte	3
1.2 Objectifs du projet	3
1.3 Périmètre du livrable 1	3
2. Contraintes générales du projet	3
2.1 Outils et environnement de travail	3
2.2 Langage et bibliothèque	3
2.3 Qualité et maintenabilité du code	3
3. Exigences du système – EasySave v1.0	4
3.1 Exigences fonctionnelles	4
3.2 Exigences non fonctionnelles	6
4. Modélisation orientée objet	7
4.1 Architecture en couches	7
4.2 Principes de conception	7
4.3 Gestion de versions et organisation GitHub	8
5. Critères de validation	8
5.1 Critères détaillés par domaine	8
6. Livrables attendus	10
6.1 Code source et exécutable	10
6.2 UML	10
6.3 Documentation	10
7. Calendrier	10
8. Glossaire	10

1. Présentation du projet

1.1 Contexte

ProSoft est un éditeur de logiciels commercialisant ses produits sous forme de Suite. Dans le cadre de l'évolution de cette suite, la direction a confié à notre équipe le développement d'un nouveau logiciel de sauvegarde nommé EasySave.

Ce logiciel s'intégrera à la politique tarifaire ProSoft existante, avec un prix unitaire de 200 € HT et un contrat de maintenance annuel représentant 12 % du prix d'achat.

1.2 Objectifs du projet

Le présent cahier des charges définit les spécifications fonctionnelles et techniques de la version 1.0 (Livrable 1) du logiciel EasySave. Cette première version constitue une application Console en C# / .NET 8, permettant à l'utilisateur de créer et d'exécuter des travaux de sauvegarde de fichiers.

1.3 Périmètre du livrable 1

Ce document couvre exclusivement EasySave version 1.0. Les versions 2.0 et 3.0 feront l'objet de cahiers des charges distincts.

2. Contraintes générales du projet

2.1 Outils et environnement de travail

L'ensemble des développements doit respecter les contraintes imposées par ProSoft et validées par le responsable d'équipe :

- IDE : Visual Studio 2022 ou version supérieure
- Versioning : GitHub (le tuteur/pilote doit être invité en tant que collaborateur)
- Éditeur UML : ArgoUML (recommandé)
- Tous les documents et codes sources doivent être gérés via ces outils

2.2 Langage et bibliothèque

- Langage : C#
- Bibliothèque cible : .NET 8.0

2.3 Qualité et maintenabilité du code

Dans une optique de reprise facilitée par d'autres équipes, les contraintes suivantes sont imposées :

- L'ensemble des documents, lignes de code et commentaires doit être exploitable par les filiales anglophones (usage de l'anglais recommandé)
- Le nombre de lignes de code dans une fonction doit rester raisonnable
- La redondance de code (copier-coller) est à proscrire
- Les conventions de nommage C# doivent être respectées
- La documentation utilisateur doit tenir en une seule page
- Une release note est obligatoire pour chaque version

3. Exigences du système – EasySave v1.0

Cette section décrit l'ensemble des exigences fonctionnelles et non fonctionnelles retenues pour la version 1.0 du logiciel EasySave.

3.1 Exigences fonctionnelles

3.1.1 Gestion des travaux de sauvegarde

Le logiciel doit permettre la création et la gestion de jusqu'à 5 travaux de sauvegarde. Chaque travail est défini par les attributs suivants :

- Un nom de travail de sauvegarde
- Un répertoire source
- Un répertoire cible
- Un type de sauvegarde :
 - Sauvegarde Complète (Full) : copie intégrale de tous les fichiers et sous-répertoires
 - Sauvegarde Différentielle (Differential) : copie uniquement des fichiers modifiés depuis la dernière sauvegarde complète

Les 5 travaux de sauvegarde sont persistés dans un fichier de configuration unique au format JSON : **%ProgramData%\EasySave\config.json**

Ce fichier contient un tableau de travaux (maximum 5).

L'application propose un menu interactif en console permettant :

- Lister les travaux existants
- Ajouter un nouveau travail (nom, source, cible, type)
- Modifier un travail existant
- Supprimer un travail
- Exécuter un ou plusieurs travaux Si aucun argument n'est passé en ligne de commande, l'application démarre automatiquement en mode interactif.

Sauvegarde Différentielle

Seuls les fichiers dont la date de dernière modification (LastWriteTime) est postérieure à la date de la dernière sauvegarde complète du même travail sont copiés. Chaque travail

stocke dans config.json la date/heure de sa dernière sauvegarde complète (LastFullBackupDate). Après une sauvegarde complète réussie, cette date est mise à jour.

3.1.2 Exécution des sauvegardes

L'utilisateur pourra exécuter les sauvegardes de plusieurs façons :

- Si l'application est lancée sans argument alors l'application passe en mode interactif via le **menu principal**.
- Si des arguments sont passés :
 - Exécuter un seul travail (ex : EasySave.exe 2)
 - Exécuter plusieurs travaux de manière séquentielle (un par un) :
 - Par plage : EasySave.exe 1-3
 - Par liste : EasySave.exe 1 ;3 ;5.

L'exécution devra être possible via l'interface console interactive ou via la ligne de commande directement.

3.1.3 Gestion des sources et destinations

Les répertoires source et cible peuvent être situés sur des disques locaux, des disques externes ou des lecteurs réseau (UNC). Tous les fichiers et sous-répertoires du répertoire source doivent être sauvegardés.

3.1.4 Fichier de log journalier

Le logiciel doit produire un fichier de log journalier au format JSON, alimenté en temps réel. Chaque entrée doit contenir :

- Horodatage
- Nom du travail de sauvegarde
- Chemin complet du fichier source (format UNC)
- Chemin complet du fichier de destination (format UNC)
- Taille du fichier
- Temps de transfert en ms (valeur négative en cas d'erreur)

Le nom du fichier log suit le format : yyyy-MM-dd.json. Un fichier est généré par jour.

Ce mécanisme de log doit être implémenté dans une DLL indépendante : EasyLog.dll, afin d'être réutilisable par d'autres projets.

3.1.5 Fichier d'état temps réel

Un fichier unique au format JSON (state.json) doit contenir, en temps réel, l'état global et individuel de chaque travail de sauvegarde. Pour chaque travail, les informations suivantes doivent être enregistrées :

- Nom du travail
- Horodatage de la dernière action
- État (Actif / Inactif / Terminé / Erreur...)
- Nombre total de fichiers éligibles
- Taille totale à transférer

- Progression (%)
- Nombre de fichiers restants
- Taille restante
- Fichier actuellement en cours de copie (source + destination)

3.2 Exigences non fonctionnelles

Langues

L'interface et tous les messages doivent être disponibles au minimum en français et en anglais, la langue étant détectée de façon automatique via la langue du système.

Chemins supportés

Les répertoires source et cible peuvent être situés sur des disques locaux, des disques externes ou des lecteurs réseau (UNC). Les emplacements temporaires de type C:\temp ou C:\Users sont strictement interdits. L'emplacement recommandé est %ProgramData%\EasySave.

Format JSON

Tous les fichiers JSON (log, état, configuration) doivent être bien indentés avec des retours à la ligne, afin d'être lisibles directement dans Notepad.

DLL EasyLog

La bibliothèque EasyLog.dll doit être développée comme un projet séparé et doit rester rétrocompatible avec la version 1.0 pour toutes les évolutions futures.

Qualité du code

- Pas de code dupliqué (principe DRY – Don't Repeat Yourself)
- Fonctions courtes (idéalement < 40 lignes)
- Respect strict des conventions de nommage C# (PascalCase pour classes/méthodes, camelCase pour variables)
- Tous les commentaires et identifiants doivent être rédigés en anglais
- Code modulaire et prêt pour l'évolution vers une interface graphique (MVVM en v2)
- Code Propre et lisible.

Performance

- Temps de copie optimisé
- Écriture rapide des logs sans bloquer le processus de sauvegarde
- Comportement acceptable sur des volumes importants de fichiers

Fiabilité

- Gestion propre des erreurs (pas de crash brutal)
- En cas d'échec de transfert, la valeur du temps est négative dans le log
- Le logiciel continue la sauvegarde des autres fichiers en cas d'erreur unitaire

Portabilité

- Compatible avec différents types de supports : local, externe, réseau
- Aucune dépendance à des chemins fixes ou spécifiques à une machine

4. Modélisation orientée objet

La conception d'EasySave v1.0 doit anticiper les évolutions futures (notamment l'interface graphique MVVM en v2) et respecter les principes SOLID, en particulier le principe de responsabilité unique (Single Responsibility).

4.1 Architecture en couches

Le projet est découpé en **quatre projets Visual Studio distincts** suivant une architecture en couches stricte :

- **EasySave.Core** : Logique métier pure (entités BackupJob, services, stratégies de sauvegarde). Ne dépend d'aucune autre couche.
- **EasySave.Infrastructure** : Accès aux ressources externes (fichiers, JSON, système de fichiers). Implémente les interfaces définies dans le Core.
- **EasySave.ConsoleApp** : Couche de présentation (menu interactif, parsing des arguments en ligne de commande). Dépend du Core.
- **EasyLog** : Bibliothèque de classe indépendante (DLL) dédiée à la journalisation, réutilisable par tous les projets ProSoft.

4.2 Principes de conception

- **Principes SOLID appliqués** :
 - **Single Responsibility Principle (SRP)** : chaque classe a une seule responsabilité.
 - **Open/Closed Principle (OCP)** : les algorithmes de sauvegarde sont extensibles sans modification (pattern Strategy).
 - **Dependency Inversion Principle (DIP)** : le Core dépend d'abstractions, pas d'implémentations concrètes.
- **Design Pattern principal : Strategy** pour les deux types de sauvegarde.
 - Interface IBackupStrategy
 - Deux implémentations : FullBackupStrategy et DifferentialBackupStrategy
 - Injection via le BackupJob (composition).
- **Autres patterns utilisés** :
 - **Repository** pour la persistance de la configuration (IConfigRepository) et de l'état (IStateRepository).

- **Singleton** (ou service statique) pour le logger EasyLog.
- **DRY** : aucune duplication de code. Les opérations communes (copie de fichier, calcul de progression, mise à jour du state.json) sont factorisées dans des services dédiés.

Préparation à l'évolution MVVM (v2.0)

Le Core est entièrement indépendant de l'interface utilisateur. Il sera donc possible, en v2, de remplacer EasySave.ConsoleApp par une couche EasySave.ViewModels sans toucher au code métier ni à l'Infrastructure.

4.3 Gestion de versions et organisation GitHub

L'ensemble du code source doit être versionné sur GitHub avec une organisation rigoureuse :

- Branches : main, develop, feature/xxx, bugfix/xxx
- Commits fréquents, atomiques et bien rédigés (en anglais)
- Solution Visual Studio bien organisée (multi-projets)
- Le tuteur/pilote doit être invité comme collaborateur sur le dépôt

5. Critères de validation

Le projet Livrable 1 sera considéré comme conforme si les critères suivants sont satisfaits. L'évaluation est répartie selon les pondérations ci-dessous :

Catégorie	Critère d'évaluation	Poids
Respect des contraintes ProSoft	Respect total des outils (VS 2026, .NET 8, GitHub, ArgoUML)	15%
Gestion de projet	Qualité du Git (branching, commits, historique)	15%
Qualité du code	Propreté, maintenabilité, absence de duplication	20%
Architecture & Conception	Séparation des responsabilités, extensibilité	15%
Fonctionnalités	Respect complet du cahier des charges fonctionnel	20%
Documentation	Manuel utilisateur (1 page), Release Notes, Support Tech	10%
Tests & Robustesse	Gestion des erreurs, chemins réseau, cas limites	5%

5.1 Critères détaillés par domaine

A. Gestion de l'environnement de travail (15%)

- Repository GitHub correctement configuré et tuteur invité en Collaborateur
- Utilisation correcte des branches (main, develop, feature/xxx)
- Commits fréquents, atomiques et bien rédigés en anglais
- Solution Visual Studio bien organisée (multi-projets)

B. Qualité du code & Maintenabilité (20%)

- Zéro duplication de code (point très surveillé par les évaluateurs)
- Fonctions courtes et lisibles (idéalement < 40 lignes)
- Respect strict des conventions de nommage C#
- Tous les commentaires et identifiants en anglais
- Code modulaire et prêt pour évolution (préparation MVVM)
- Utilisation correcte de la DLL EasyLog.dll (projet séparé)
- Propreté du code

C. Architecture logicielle (15%)

- Bonne séparation en couches (Core / Infrastructure / Console / EasyLog)
- Respect des principes SOLID (au minimum Single Responsibility)
- Utilisation pertinente de Design Patterns (ex : Strategy pour Full/Differential)
- Facilité de maintenance et d'évolution vers une version graphique

D. Fonctionnalités requises (20%)

- Gestion de 5 travaux de sauvegarde maximum
- Sauvegarde Complète et Différentielle fonctionnelles
- Exécution du menu interactif par défaut
- Exécution via ligne de commande (1-3, 1;3, etc.)
- Logs journaliers JSON (format UNC, temps de transfert, etc.)
- Fichier state.json mis à jour en temps réel
- Support chemins locaux et réseau
- Interface console bilingue (FR/EN) avec détection automatique de la langue système.

E. Documentation (10%)

- Manuel utilisateur : 1 seule page (clair, professionnel, bilingue)
- Release Notes détaillées
- Fiche Support Technique (emplacements fichiers, configuration minimale, prérequis)
- README GitHub complet

F. Aspects techniques spécifiques (5%)

- Fichiers JSON bien indentés et lisibles
- Emplacements professionnels (%ProgramData%\EasySave\...)
- Gestion des erreurs propre (pas de crash brutal)
- Performances raisonnables sur gros dossiers

6. Livrables attendus

6.1 Code source et exécutable

- Code source complet versionné sur le dépôt GitHub
- Application Console compilée (.exe) et sa DLL EasyLog

6.2 UML

- Diagrammes de classes
- Diagramme de cas d'utilisation
- Diagrammes de séquence
- Livrés 24h avant le jour du livrable (La Veille)

6.3 Documentation

- User Manual : manuel d'utilisation en une page (FR/EN)
- Technical Doc : emplacements par défaut, prérequis, configuration minimale
- Release Note : historique des versions et corrections

7. Calendrier

Échéance	Action
Jour 1	Lancement du projet et rédaction du Cahier des Charges v1.0
Jour 3	Création de l'environnement de travail + envoi des accès tuteur (évalué sur tout le projet)
Veille Livrable 1	Livraison des diagrammes UML
Jour Livrable 1	Réception du livrable 1 (EasySave v1.0) + documentations associées

8. Glossaire

Terme	Définition
Travail de sauvegarde	Unité de configuration définissant une opération de copie entre un répertoire source et un répertoire cible.
Sauvegarde complète	Copie intégrale de tous les fichiers du répertoire source vers le répertoire cible.
Sauvegarde différentielle	Copie uniquement des fichiers modifiés depuis la dernière sauvegarde complète.
Format UNC	Universal Naming Convention : format de chemin réseau (ex : \\serveur\dossier\fichier).

EasyLog.dll	Bibliothèque dynamique développée par l'équipe, dédiée à la journalisation des actions de sauvegarde.
State.json	Fichier JSON unique enregistrant en temps réel l'état de progression de chaque travail de sauvegarde.
DLL	Dynamic Link Library – bibliothèque de fonctions réutilisables chargée dynamiquement par le programme.
.NET 8	Version de la plateforme de développement Microsoft utilisée pour ce projet.